

Avaliação 03

Descritivo de Implementação

O presente documento tem por objetivo servir de guia para a elaboração dos trabalhos da TERCEIRA unidade da componente curricular GRAFOS. Ao fim desta unidade, é esperado que o aluno tenha conhecimento sobre a teoria relativa ao conteúdo abordado, e consiga replicá-la em um ambiente computacional.

Como ficou definido na aula inicial, as atividades avaliativas irão constar de implementações computacionais dos pressupostos teóricos vistos em sala de aula. Tais implementações deverão seguir algumas regras básicas.

- (a) As implementações deverão ser feitas em grupo (o mesmo que está sendo considerado desde a primeira atividade);
- (b) A linguagem a ser adotada é de livre escolha. Contudo, as implementações devem abranger a teoria vista integralmente. O uso de bibliotecas auxiliares é permitido, porém, não para as atividades fim dos algoritmos. Em outras palavras, as estruturas de dados e pseudocódigos devem ser implementados em sua integralidade;
- (c) Apesar de ser uma atividade em grupo, a avaliação ocorrerá de forma individual, de forma que todos devem ter conhecimento dos códigos e funções implementadas pelo grupo.
- (d) Não custa lembrar que a presença física nas aulas também é objeto de avaliação, contribuindo para a construção da nota individual. Então, evitem faltar.
- (e) A data **limite** para entrega e apresentação (**VIRTUAL**) é o dia **07 de julho de 2026**.
- (f) Os grupos também devem preencher os resultados encontrados na planilha compartilhada, na guia "*resultados*" (no caso de escolha do tema relacionado ao Problema do Caixeiro Viajante);
- (g) Lembro que a quantidade máxima de faltas deve obedecer ao limite definido institucionalmente, pois também é um critério **necessário** para a aprovação na disciplina.

Com relação ao trabalho, deverá ser entregue:

- i. **O código utilizado, para inspeção:** Os códigos deverão estar comentados, **em português**, detalhando o funcionamento de cada função/procedimento, incluindo o formato da entrada e qual a saída esperada. (Poderá ser disponibilizado em GitHub ou similar)
- ii. **Um relatório**, descrevendo as funções desenvolvidas (como gerou a população inicial, quais foram as buscas locais e como foram implementadas, como efetuou a escolha dos parâmetros dos algoritmos genéticos e meméticos), a linguagem de programação utilizada, detalhes do hardware e do compilador, bem como a comparação dos resultados obtidos com os melhores resultados constantes na planilha compartilhada.
- iii. Um vídeo curto (10~15min), explicando o funcionamento/estrutura geral do programa, IDE utilizada e os resultados decorrentes da aplicação da função aos dados da avaliação.
- iv. **Uma lista de atividades desenvolvidas por integrante**, detalhando a participação efetiva em cada implementação, seja em sua concepção, implementação, revisão ou teste.

O tema a ser escolhido é de livre escolha, dentre os dois a seguir:

A. PROBLEMA DO CAIXEIRO VIAJANTE (PCV)

O Problema do Caixeiro-Viajante (PCV) é um dos problemas mais estudados da otimização combinatória e consiste em determinar um ciclo de custo mínimo que visite cada vértice de um grafo exatamente uma vez, retornando ao vértice de origem. Em sua formulação clássica, os vértices representam localidades e as arestas representam os custos de deslocamento, como distância, tempo ou custo financeiro. O objetivo é minimizar o custo total da rota, garantindo que todos os vértices sejam visitados uma única vez. Para este estudo, vamos considerar a problemática específica descrita no artigo compartilhado.

B. ORIENTEERING PROBLEM (OP)

O *Orienteering Problem* é um problema clássico de otimização combinatória que combina características do problema do caixeiro-viajante com a coleta de bônus. Seu objetivo é determinar uma rota entre um vértice de origem e um vértice de destino, respeitando **um limite máximo de custo** (como distância, tempo ou orçamento), de modo a maximizar a soma das recompensas associadas aos vértices visitados. Diferentemente do problema do caixeiro-viajante, não é necessário visitar todos os vértices da rede; em vez disso, o algoritmo deve selecionar estrategicamente apenas aqueles cuja inclusão proporcione o maior benefício sem violar a restrição de custo. Note que, em vez de encontrar o custo MÍNIMO, neste problema, deve-se MAXIMIZAR a coleta das recompensas, respeitando um limite de custo, associado à trafegar pelas arestas. Considere a formação de um ciclo, saindo da cidade inicial (primeira do arquivo).

Para ambos os temas, deverá ser implementado:

- (1) Heurística do Vizinho mais Próximo + Busca Local
- (2) Heurística da Inserção mais barata + Busca Local
- (3) Algoritmo Genético
- (4) Algoritmo Memético com pelo menos 3 buscas locais.
- (5) Para os algoritmos genéticos e meméticos, deverão ser executadas 20 iterações do algoritmo **para cada instância**. Um arquivo em formato TXT contendo o resumo dos resultados obtidos deve ser gerado. Deve ser destacado o menor valor encontrado entre as 20 execuções, o valor médio e o tempo médio de execução.

Os dados a serem utilizados, bem como a descrição dos problemas estão disponíveis na pasta https://drive.google.com/drive/folders/1y_UWdgzWKet9Ue7KpmHhZGfuwpS5jrS-?usp=sharing

Para o PCV, existe um artigo base, denominado GRASP_SBP02025.pdf, que descreve os cenários a serem analisados. São 12 problemas sendo 6 por distância e 6 por tempo. Está anexado também uma planilha com os dados.

Para o *Orienteering Problem*, são listados dois artigos, em inglês e um artigo compactado com os dados, disponíveis no arquivo "OP_dataset.zip". O **limite máximo de custo** é o primeiro dado do arquivo, e para cada linha, temos três informações. As duas primeiras são as coordenadas "X" e "Y" do ponto, e a terceira informação é a "recompensa" referente ao ponto. Seja $i=(x_i, y_i)$ e $j=(x_j, y_j)$ dois vértices da instância. A distância euclidiana entre eles é dada por:

$$d_{i,j} = [(x_i - x_j)^2 + (y_i - y_j)^2]^{0.5}$$

Deve-se calcular a matriz de distâncias $D=[d_{ij}]$, calculando a distância entre todos os pontos.

Obs. As técnicas de busca local, quanto utilizadas como pós-processamento das heurísticas do Vizinho mais Próximo e da Inserção Mais Barata, são algoritmos determinísticos. Logo, para estes casos basta executar uma única vez. A escolha de quais buscas locais implementar é livre.